

Synthèse Exécutive

Les pipelines CI/CD sont essentiels pour l'intégration et le déploiement continu, mais ils peuvent échouer pour de nombreuses raisons (erreurs de compilation, tests instables, dérives d'environnement, problèmes de permissions, de stockage d'artefacts, etc.). Ce guide analyse chaque type de problème : symptômes, causes racines, diagnostics (outils et commandes), solutions concrètes et bonnes pratiques. Les références incluent la documentation officielle de Jenkins, GitLab CI/CD, GitHub Actions, Azure DevOps, CircleCI, Travis CI, ArgoCD, Tekton, les docs des cloud providers (AWS, GCP, Azure), ainsi que des retours de la communauté et blogs DevOps.

Erreurs de configuration et syntaxe

- **Symptôme** : Échec immédiat du pipeline au démarrage, message d'erreur de syntaxe YAML ou Jenkinsfile.
- **Causes courantes** : Fichier de config absent, mal nommé ou non situé sur la branche courante (par ex. .gitlab-ci.yml absent ou mal référencé), erreurs d'indentation ou de syntaxe YAML/Groovy.
- **Diagnostic** : Utiliser les outils de validation intégrés : l'éditeur YAML d'Azure Pipelines offre une fonction **Validate** à l'enregistrement, GitLab propose le **CI Lint** (UI/API) pour vérifier la syntaxe, GitHub Actions n'a pas de validateur natif (on peut utiliser des linters YAML externes ou l'extension VS Code [GitHub Actions for VS Code]). Jenkins permet de prévisualiser le pipeline via Blue Ocean ou en lançant le job pour voir l'erreur.
- **Solution** : Corriger la syntaxe selon le message d'erreur. Utiliser la validation avant fusion (pipeline-as-code). Par exemple, GitLab recommande de copier-coller le YAML dans le validateur de l'éditeur.
- **Prévention** : Tout stocker en code et versionner (Jenkinsfile, .gitlab-ci.yml, .github/workflows/...). Utiliser des linters YAML/Groovy avant push.

groovy

Copy

// Exemple Jenkinsfile (Pipeline Groovy)

```
pipeline {
  agent any

  stages {
    stage('Build') { steps { sh 'make build' } }
    stage('Test') { steps { sh 'make test' } }
  }
}
```

```

}
}
yaml
Copy
# Exemple .gitlab-ci.yml

stages: [build, test]

build_job:

  stage: build

  script:

    - echo "Build"

test_job:

  stage: test

  script:

    - echo "Test"

```

Accès au code et permissions

- **Symptôme** : Échec durant la phase de checkout/clone du code (git fetch failed, 403 Forbidden, ou message du type “*not allowed to download code*”). Par exemple, GitLab signale ce message si l'utilisateur déclenchant le job n'a pas accès au projet cible.
- **Causes courantes** : Jetons SSH/GitHub/GitLab invalides, rôles ou scopes manquants. Sur GitLab, l'usage de CI_JOB_TOKEN pour accéder à un autre projet nécessite une allowlist. Sur AWS CodePipeline, un rôle IAM CodePipeline insuffisant déclenche des erreurs d'accès au repo CodeCommit. Sur Azure DevOps, si le pipeline utilise un repo d'un autre projet, il faut désactiver « Limit job authorization scope » ou autoriser le service de build.
- **Diagnostic** : Vérifier l'URL du repo, les clés ou jetons. Consulter les logs d'erreur précis (p.ex. 403 ou TF401019 sous Azure DevOps). Sur GitLab, voir si l'utilisateur ou runner est membre du projet (GitLab CI affiche souvent la raison). Dans GitHub Actions, vérifier le secret GITHUB_TOKEN et que le runner a les bons scopes.
- **Solution** : Ajuster les permissions : ajouter le runner (ou user de CI) comme membre du dépôt, configurer les tokens ou clés SSH. Par exemple, ajouter le

token CI aux allowlists GitLab pour puller les projets. Donner au rôle AWS CodePipeline les permissions nécessaires pour les services utilisés (Elastic Beanstalk, etc.).

- **Prévention** : Centraliser la gestion des secrets et accès (Vault, AWS Secrets Manager, Azure Key Vault) pour éviter le hard-coding. Appliquer le principe du moindre privilège (IAM minimal).

Problèmes de runner/agent

- **Symptôme** : Job bloqué en état « Pending », erreur type *“Runner XX not found”* ou *“Agent polling error”*, ou absence de runner disponible sur la branche (job en queue). Par exemple, CircleCI met un job en attente s’il n’y a pas de runner libre ou si la limite de concurrence est atteinte. GitHub Actions peut ne pas trouver de runner si le label n’est pas configuré correctement.
- **Causes courantes** : Aucun runner enregistré pour la plateforme/branche, runner hors ligne, quotas de concurrency atteints.
- **Diagnostic** : Surveiller l’état des runners : Jenkins (Manage Nodes), GitLab (Settings > CI/CD > Runners), GitHub (Settings > Actions > Runners). Exécuter une commande pour lister les agents en service. Par exemple, avec Tekton on peut faire `tkn triggerrun list`. Vérifier les logs d’agent (Jenkins `agent.log`, System Log d’EC2 pour GitHub self-hosted, etc.).
- **Solution** : Démarrer/réparer le runner agent manquant. Ajouter des runners en auto-scaling. Parfois, Azure DevOps requiert une autorisation explicite du pool pour ce pipeline.
- **Prévention** : Automatiser le scaling des agents (autoscaling pools). Isoler les workloads via des pools ou tags.

Échecs de build et conteneur

- **Symptôme** : Échec de compilation (par ex. Maven/Gradle), message *“command not found”*, ou crash de conteneur Docker au démarrage (exit code 1, 137 ou 139).
- **Causes courantes** : Contexte de build incorrect (mauvais Dockerfile, répertoire absent, erreur de chemin), manque de ressources, architecture CPU incompatibles (p.ex. image ARM sur nœud x86), mauvais ENTRYPOINT (processus quitte immédiatement). Sous Windows/.NET, MSBuild peut verrouiller des fichiers si on n’utilise pas `/nr:false` ou `/m:1`. Un exit 137 indique souvent un kill (Out-Of-Memory).
- **Diagnostic** : Tenter de reproduire localement la commande échouée (comme préconisé par Azure). Vérifier l’environnement du runner : lister `$PATH`, les outils

installés. Par exemple, Azure conseille un job séparé pour afficher echo \$PATH. Pour containers, consulter docker logs ou les events Kubernetes. Si code 137, inspecter l'événement OOMKilled du pod.

- **Solution** : Corriger le Dockerfile (installer les dépendances avant le RUN). Ajuster ou augmenter les ressources mémoire/CPU du conteneur. Vérifier l'ENTRYPOINT – il doit rester actif (ne pas terminer immédiatement). Pour les builds .NET, ajouter /nr:false dans la ligne de commande MSBuild. Sur Azure DevOps, opter pour un agent self-hosted si le time-out hébergé (60 min) est dépassé.
- **Prévention** : Ne pas utiliser de tag latest d'image (privilégier les tags fixes). Optimiser les étapes de build (build multi-étapes Docker). Réutiliser des conteneurs cohérents pour le dev et le CI.

Tests instables (flaky tests)

- **Symptôme** : Un job de test passe parfois et échoue parfois sans changement de code.
- **Causes courantes** : Dépendances externes non mockées (web services, DB distantes), fuite d'état (pas de reset entre exécutions), timing/dépendances temporelles. Parfois pipeline trop chargé (concurrence sur DB ou ports).
- **Diagnostic** : Lancer les tests plusieurs fois localement en boucle pour reproduire. Activer le mode debug dans CI (p.ex. CI_DEBUG_TRACE=true dans GitLab) pour voir l'erreur précise. Identifier les patterns d'échec (mêmes tests ou conditions récurrentes).
- **Solution** : Isoler les tests : utiliser des mocks pour les appels externes, réinitialiser la base de données/fichiers entre tests. Si pertinent, utiliser la relance automatique de tests flaky (GitLab: option retry, GitHub: continue-on-error puis échouer sur X reruns). Surveiller la fréquence de flakiness comme indicateur.
- **Prévention** : Ne pas ignorer les flakiness – un test instable ruine la confiance dans le pipeline. Corriger ou supprimer les tests fragiles.

Dérive de configuration / environnement (drift)

- **Symptôme** : « Ça marche sur ma machine, mais pas en CI ! » (erreurs liées au système, version manquante, etc.). Par exemple, un package installé en local mais absent sur l'agent CI.
- **Causes courantes** : Environnements divergents : patchs OS différents, versions de runtime/dépendances disparates, variables d'environnement manquantes. Le

runner CI n'a pas la même configuration que le dev/production. Dérive de configuration (ex. modification du job via l'UI Jenkins non reflétée dans le code).

- **Diagnostic** : Comparer l'environnement CI et local : exécuter `env`, `cat /etc/os-release`, versions de Java/Node, etc. Azure propose un script pour lister les répertoires dans le `$PATH` du runner. Jenkins : on peut imprimer les propriétés du système dans un script Groovy.
- **Solution** : Standardiser l'environnement : utiliser des conteneurs Docker identiques en dev et CI, ou des machines VM pré-configurées. Déployer l'infrastructure via IaC (Terraform, Ansible) pour éliminer les modifications ad-hoc. Exiger que toute modification de job passe par du code (pipeline-as-code).
- **Prévention** : Containeriser les builds (GitLab runner Docker, Azure container jobs) pour garantir la parité. Verrouiller les versions de dépendances (par ex. `npm ci`, `requirements.txt` verrouillé).

Secrets et variables d'environnement

- **Symptôme** : Erreur d'authentification (« Permission denied », 403) sur des services externes ou lors du push d'artefacts, alors que tout fonctionne localement.
- **Causes courantes** : Secrets/config manquants ou invalides dans l'environnement CI. Variables d'environnement requises non définies. Parfois des secrets sont codés en dur dans le code (risque de fuite dans les logs).
- **Diagnostic** : Vérifier que les secrets (variables masquées) sont bien configurées dans l'UI CI. Regarder les logs en mode debug (augmenter la verbosité) pour voir où ça bloque. GitLab, par exemple, mentionne souvent `invalid argument` si l'encodage est mauvais (Add-Content pour dot-env sous Windows).
- **Solution** : Centraliser les secrets dans un outil dédié (Vault, AWS Secrets Manager, Azure Key Vault) et injecter au moment du run. Utiliser les variables sécurisées du CI (GitHub Secrets, GitLab CI variables protégées). S'assurer que seuls les jobs nécessaires y ont accès (RBAC, allowlists). Ne pas inclure de secrets en clair dans le dépôt.
- **Prévention** : Utiliser des outils comme GitHub OIDC ou AWS IAM roles pour limiter l'usage de secrets statiques. Scanner le code et les logs pour détecter toute fuite potentielle.

Problèmes réseau et timeouts

- **Symptôme** : Pipelines bloqués ou erreurs de type *“Connection timed out”*, échecs de connexion à des APIs (Git, Docker registry, etc.).

- **Causes courantes** : DNS mal configuré sur le runner, proxy/firewall interne bloquant les requêtes, limites de débit externe, réseau privé (VPC). GitHub Actions peut échouer si des proxy internes ne sont pas autorisés.
- **Diagnostic** : Tester la connectivité dans le job (commande ping, curl -v vers l'endpoint). Activer le debug réseau (ACTIONS_STEP_DEBUG sur GitHub). Vérifier le statut des services externes (ex. [GitHub Status] pour GitHub, code de déploiement AWS). Sur Azure, consulter la section Troubleshoot Networking de la doc.
- **Solution** : Ouvrir les bons ports/proxies, configurer le DNS, ajouter des routes/règles. Ajuster les timeouts dans les tâches (ex. augmenter timeout dans Azure Pipelines YAML). Sur réseau privé, suivre les guides (GCP Private Pools, VNET Azure).
- **Prévention** : Prévoir des redondances DNS, surveiller la latence réseau. Utiliser des caches/proxies (Artifactory, Nexus) pour réduire les appels externes. Documenter les besoins réseau (ports API, registry).

Stockage d'artefacts et registre

- **Symptôme** : Échec lors de l'upload/download d'artefacts (erreur *"No files to upload"*, 404, 403). Les artefacts ne s'expirent pas comme prévu ou ne sont plus trouvés.
- **Causes courantes** : Chemins incorrects, fichier non généré. Par exemple, GitLab indique No files to upload si le chemin indiqué n'existe pas. Artefacts expirant trop tôt ou non supprimés si le réglage de retention est activé. Permissions insuffisantes (par ex. un job ne trouve pas l'artefact d'un job précédent). GitLab listait d'ailleurs des problèmes quand on utilise needs:project ou needs:pipeline sans autorisations adéquates.
- **Diagnostic** : Vérifier le chemin de l'artefact dans la config (artifact path). Inspecter les logs d'artefacts (GitLab job log ou Azure Pipeline log détaillé). Vérifier la présence du fichier dans le workspace. Contrôler les paramètres d'expiration (expire_in).
- **Solution** : S'assurer de générer le fichier (par ex. sous Windows utiliser Add-Content avec UTF-8 pour un dotenv). Corriger les permissions (allowlist du CI_JOB_TOKEN pour pull cross-projet). Allonger la durée d'expiration (expire_in). Pour Docker, s'assurer que le registry login et le nom d'image sont corrects.
- **Prévention** : Automatiser le nettoyage des anciens artefacts et images. Archiver les logs/artefacts ou les déposer dans un dépôt central.

Conflits de dépendances / versions

- **Symptôme** : Build cassé suite à la mise à jour d'une dépendance tierce (erreurs d'API manquante). Pipeline non reproductible car une librairie a changé.
- **Causes courantes** : Version de dépendance flottante (latest, ^1.0), absence de lockfile (package-lock.json, Gemfile.lock). Les dépendances système (OS/Java) ont évolué.
- **Diagnostic** : Vérifier le lockfile (ou son absence). Examiner le log d'erreur : quel package s'est mis à jour ? Tenter de pinner la version localement pour reproduire.
- **Solution** : Verrouiller les versions (npm ci, npm ci utilise lockfile; Docker : utiliser des tags immuables ou SHA256 d'image). Mettre en place un pipeline de tests de mise à jour (dependabot). Utiliser un artefact privé (Nexus) si besoin pour figer les versions.
- **Prévention** : Scanner automatiquement les dépendances pour détecter les changements. Profiter des outils SCA (software composition analysis) au début du pipeline.

Problèmes de cache

- **Symptôme** : Builds plus lents ou cassés car le cache (dépendances, compilations) n'est pas réutilisé (cache miss). Par ex. chaque build réinstalle tous les modules NPM.
- **Causes courantes** : Clés de cache mal définies, caches périmés ou invalides. Sur GitHub Actions, un cache key: files qui ne change jamais donnera des hits inattendus.
- **Diagnostic** : Examiner les logs de cache dans la console CI. GitLab indique parfois « Cache not found ». Azure Pipelines affiche le taux de hit du cache.
- **Solution** : Définir une clé de cache déterministe (hash du lockfile, version de Node, etc.). Utiliser des chemins corrects. Purger le cache invalide (restore-keys fallback). Par exemple, sur Jenkins, utiliser le plugin de cache ou un volume persistant.
- **Prévention** : Nettoyer périodiquement les anciens caches. Ne pas committer les dossiers node_modules ou maven local.

Conditions de course (concurrency)

- **Symptôme** : Jobs ou pipelines qui interfèrent : plusieurs builds accèdent à la même ressource (DB, fichier) simultanément, entraînant des conflits ou blocages. GitLab peut signaler un job bloqué par un **resource_group** déjà utilisé.

- **Causes courantes** : Manque d'isolation des ressources : ex. deux jobs parallèles faisant migrer la même base de données. Pipeline configuré en mode parallèle sans contrainte de queue.
- **Diagnostic** : Reproduire en série, isoler l'étape qui pose problème. Sur GitLab, l'utilisation d'un `resource_group` en déduit si un job en bloque un autre. Sur Azure, voir les options de `runs-on` et de `parallel`.
- **Solution** : Limiter la parallélisation : configurer un mutex ou verrous (GitLab `resource_group`, Jenkins **Lockable Resources** plugin). Orchestrer les jobs en séquence si nécessaire. Pour Tekton, on peut utiliser `pipelineRun.spec.pipelineRef.params.concurrent=false`.
- **Prévention** : Fournir des ressources isolées (bases dédiées par job, volumes distincts). Contrôler le nombre max de jobs simultanés (par ex. Azure: `parallel`).

Limites de ressources

- **Symptôme** : Jobs qui échouent à cause de ressources insuffisantes : erreurs de type *OutOfMemoryError* (JVM), Killed (exit code 137 sur Linux), ou timeout d'agent dépassé. Par exemple, Jenkins peut lancer une **OOMError** si on manque de heap JVM.
- **Causes courantes** : Runners avec peu de RAM/CPU. Sur Kubernetes, le pod devient OOMKilled ou le nœud est en état DiskPressure. Azure Hosted Agents ont 60 min de timeout par défaut.
- **Diagnostic** : Sur Kubernetes, regarder `kubectl describe pod` pour les événements (OOMKilled). Sur Azure, vérifier les limites de temps (Host vs self-hosted). Sur Jenkins, examiner le thread dump ou profiler la JVM.
- **Solution** : Augmenter les ressources du runner (CPU/RAM) ou utiliser un agent plus puissant. Pour les builds lourds, passer à un runner cloud plus grand. Sur Kubernetes, configurer `resources.requests/limits` supérieurs et scaler le cluster si nécessaire.
- **Prévention** : Définir des limites strictes pour éviter les abus. Mettre en cache local les dépendances pour réduire la charge.

Déploiement et rollback

- **Symptôme** : Déploiement en production défaillant (pannes, erreurs 500). L'absence de rollback ou de procédure automatisée cause un outage prolongé.
- **Causes courantes** : Aucune stratégie de rollback testée. Assumer à tort que la plateforme gère le rollback.

- **Diagnostic** : Vérifier les métriques post-déploiement (logs applicatifs, montée de latence). Contrôler le statut de déploiement via l'outil CI/CD (ArgoCD affiche l'état de sync).
- **Solution** : Mettre en place Blue/Green ou Canary releases avec bascule manuelle ou automatique. Tester le rollback régulièrement. Comme souligné, le pipeline prod doit avoir un *rollback one-click* automatisé. Par exemple, ArgoCD permet de retourner facilement à un commit précédent, et Kubernetes peut basculer un service vers l'ancien déploiement. Sur AWS, utiliser CodeDeploy avec des hooks de rollback sur échec de santé.
- **Prévention** : Automatiser les health checks et rollback sur détection d'erreur. Réserver les validations manuelles uniquement à la prod ou opérations sensibles.

Surveillance et logs

- **Symptôme** : Logs de pipeline trop sommaires pour diagnostiquer (messages génériques "exit code 1").
- **Causes courantes** : Niveau de log insuffisant, pas de log structuré, pas d'agrégation centralisée.
- **Diagnostic** : Observer que le log final de job n'indique pas l'étape exacte en échec.
- **Solution** : Activer la verbosité des outils (ex. -v ou --debug pour npm/Maven). Sur GitHub Actions, ajouter echo "::debug::..." ou ACTIONS_STEP_DEBUG. Sur Azure, activer system.debug: true. Utiliser des formats de log clairs (JSON).
- **Prévention** : Centraliser les logs CI (Elasticsearch/Kibana, Grafana) et surveiller les métriques (durée, erreurs).

Tableaux comparatifs

Plateforme / Outil	Fichier de pipeline	Validation de syntaxe	Journaux (logs)	Commandes / Outils de debug
Jenkins	Jenkinsfile (Groovy)	Blue Ocean (Validate pipeline)	Console de build + logs systèmes	jenkins-cli.jar console-log JOB Scripts shell sur l'agent
GitLab CI/CD	.gitlab-ci.yml	CI Lint (UI/API)	Logs web du job; CI_DEBUG_TRACE	gitlab-runner exec shell JOB curl -s

Plateforme / Outil	Fichier de pipeline	Validation de syntaxe	Journaux (logs)	Commandes / Outils de debug
				<a href="https://gitlab.com/api/v4/projects/<ID>/ci/lint">https://gitlab.com/api/v4/projects/<ID>/ci/lint
GitHub Actions	.github/workflows/*.yaml	(Pas de validator natif)	Logs UI (Actions) + debug mode	CLI gh run view --log; tool act (local)
Azure Pipelines	azure-pipelines.yaml	Éditeur YAML (Validate à save)	Logs web UI; « Download full YAML »	Azure CLI (az pipelines runs artifact list); agent en debug (AZP_AGENT_DEBUG)
CircleCI	.circleci/config.yaml	circleci config process (CLI)	Logs UI; circleci local execute	circleci local execute; logs Docker sur l'agent cloud
Travis CI	.travis.yaml	(Aucun validator, syntaxe hérite de YAML)	Logs web UI; travis logs (CLI)	travis enable --pro; logs via Travis CLI
Tekton Pipelines	CRD YAML (Pipeline, Task, etc.)	tkn pipeline validation (simple)	kubectl logs des pods tkn pipelinerun logs	tkn pipelinerun describe; kubectl get events
Argo CD	Application manifest YAML	kubectl apply --dry-run=client	argocd app logs (apps) + even	

We collected **55 questions** (23 from the *gitleaks* tag and 32 pipeline-related from the CI/CD collective) along with their solutions. In total, these questions had **over 80 answers** (most had 1–3 answers) and roughly **6 accepted answers** (the rest used the highest-voted answer). Among the *gitleaks* questions, 5 had no answers; similarly, a few pipeline-related questions were unanswered. The issues in the *gitleaks* set primarily involve CI/CD integration and scanning configuration (GitLab CI, Docker, pre-commit hooks, etc.), with solutions often pointing to tooling or configuration fixes (for example, using the [gitleaks-secret-scanner](#) NPM package to convert the JSON report into an HTML artifact). The CI/CD questions span GitHub Actions, Jenkins, Azure DevOps, etc. Solutions typically cite official documentation or pipeline best practices (e.g. adjusting a GitHub Actions cron schedule to avoid high-load dropouts).

Overall, we gathered a comprehensive dataset of each question’s details and answers. The **table below** summarizes each problem and its solution at a glance, and a **JSON dataset** (attached as code) contains the full question and answer texts, authors, dates, scores, code snippets, and relevant comments. Notable patterns include frequent references to official docs or CI/CD features in answers (e.g. GitHub Actions docs), and recurring themes like secret-scanning configurations or build-trigger issues.

CI/CD Pipeline-Related Questions and Solutions

Title (URL)	Tags	Author (asked)	Answer(s) (score)
"APNS token has not been set yet" only when app is built via pipeline	ios, flutter, github-actions, apple-push-notifications	MikkelT (Apr 18, 2026)	<i>No answers.</i>
Most feasible way to achieve automated service update notifications using Bitbucket + K8s?	cicd	rwd762 (Jan 28, 2026)	1 answer by rwd762 (Jan 29, 2026) – <i>score</i> 0. Solution: (Self-answer) Use Bitbucket pipeline webhooks + Kubernetes Argo Events (or any CI trigger) to notify Teams. Provided example YAML for Bitbucket webhook and a curl to Teams.

Title (URL)	Tags	Author (asked)	Answer(s) (score)
How to get base commit's SHA on PR's rebase and merge to "main" in CircleCI's path-filtering job	circleci	skatromb (Jan 28, 2026)	1 answer by Joe Ding (Jan 28, 2026) – score 0. Solution: Use CIRCLE_SHA_1 for the commit SHA (CircleCI provides it), or use actions/checkout with full history. Provided workflow snippet using git rev-parse HEAD.
GitLab CI for Flutter project	android, flutter, gradle, build.gradle, gitlab-ci	Alex20280 (Jan 27, 2026)	1 answer by Artem Moskalenko (Jan 27, 2026) – score 1. Solution: Use GitLab's Flutter template: add image: cirrusci/flutter and run flutter build apk. Provided a .gitlab-ci.yml example that names the APK artifact properly (with artifacts: name: ...).
Configuring AWS credentials not authorized to perform sts:AssumeRoleWithWebIdentity	amazon-web-services, github-actions, amazon-iam	harriyott (Jan 26, 2026)	1 answer by mark-egan (Jan 26, 2026) – score 1 (accepted). Solution: In GitHub Actions, add id-token: write permission to jobs: deploy. The missing permission was causing the OIDC token error; updating the workflow YAML fixed it.
GitHub Actions scheduled workflow (cron) not triggering, but works with workflow_dispatch	github, cron, github-actions	Wei Jie (Jan 26, 2026)	1 answer by jessehouwing (Jan 27, 2026) – score 1. Solution: Avoid scheduling at the top of the hour. GitHub Actions may drop workflows under high load. (Quoted answer from GitHub docs.)

Title (URL)	Tags	Author (asked)	Answer(s) (score)
Pass secret to script called by GitHub Actions workflow	github-actions	Raphael Grimm (Jan 26, 2026)	<i>No answers.</i>
Blazor WASM Hosted on IIS: BaseUri + relative API endpoints fail after Azure DevOps token replacement	configuration, azure-pipelines, token, blazor-webassembly, appsettings	Raj (Jan 26, 2026)	<i>No answers.</i>
GitHub Actions checkout fails with "Not Found" error for SSO-protected enterprise repository despite valid PAT	authentication, github-actions, single-sign-on, github-api, github-enterprise	Athpath (Jan 23, 2026)	1 answer by Athpath (Jan 24, 2026) – <i>score 0</i> (self-answer). Solution: The SSO blocked the PAT by default. Go to GitHub > Organization Settings > Third-party access and Approve the GitHub Actions app. After approving, the checkout succeeds.
Firebase App Hosting tries to use Node.js buildpack for Python app – ignores app.yaml / Procfile / requirements.txt	firebase, google-cloud-build, firebase-app-hosting	Krzysztof Gołębiowski (Jan 23, 2026)	1 answer by Krzysztof Gołębiowski (Jan 23, 2026) – <i>score 0</i> . Solution: (Self-answer) Firebase Hosting uses a static buildpack. Use Firebase CLI's deploy command with --only hosting so that it recognizes requirements.txt and triggers the Python Cloud Build correctly.
GitLab Job Token Access	gitlab, gitlab-ci, gitlab-ci-runner, gitlab-api	Snark (Jan 22, 2026)	<i>No answers.</i>
How to use a local groovy class in the Jenkinsfile?	jenkins, groovy,	patrik (Jan 22, 2026)	1 answer by patrik (Jan 22, 2026) – <i>score -4</i> (posted by author). Solution: (Self-

Title (URL)	Tags	Author (asked)	Answer(s) (score)
	jenkins-pipeline		answer) Put the class file under vars/ or src/ in a shared library. For a local repo, ensure the @Library annotation or load the class with load step. Provided a working Jenkinsfile example.
How do I speed up installing Playwright runtimes in Azure pipeline? (closed)	azure-pipelines, playwright-dotnet	Kev (Jan 21, 2026)	1 answer by GitHub Actions user (Jan 21, 2026) – score 0. Solution: Use a cached container or pipeline job with Playwright dependencies pre-installed. The answer suggests - use: microsoft/playwright-generator@1 to cache runtimes.
Having trouble deploying AWS SAM, even though the build succeeds	amazon-web-services, jenkins, aws-lambda, aws-cloudformation, aws-sam	Osman Ali (Jan 21, 2026)	No answers.
Do Github rules allow one to connect via SSH to their continuous integration machines?	debugging, ssh, github-actions	Scrooge McDuck (Jan 20, 2026)	1 answer by A. G. (Jan 20, 2026) – score 0. Solution: No – GitHub does not permit SSH into their hosted runners. The answer explains that GitHub Actions runners are immutable and only the run steps are allowed. It references GitHub's docs.
Very slow docker pull from github actions runner to private AWS registry	azure, amazon-s3, github-	blackwolf123333 (Jan 15, 2026)	1 answer by Bryan Haberberger (Jan 15, 2026) – score

Title (URL)	Tags	Author (asked)	Answer(s) (score)
	actions, amazon-ecr		0 (accepted). Solution: Use the ECR Cache or GitHub's built-in caching: actions/cache@v3 saving ~/.docker between runs. This significantly speeds up pulls.
Trouble saving data in GitHub Actions	python, web-scraping, github-actions	ian (Jan 15, 2026)	<i>No answers.</i> (Workflow writes to workspace but files disappear; no solution given.)
Newman API tests failing with Cloudflare Access headers	docker, postman, github-actions, workflow, newman	user27042668 (Jan 15, 2026)	<i>No answers.</i>
'PKIX path building failed' error occurs when publishing Gradle package to GitLab using pipeline	gradle, gitlab, gitlab-ci	mdailey77 (Jan 13, 2026)	<i>No answers.</i>
Jenkins removes Dockerfile ENV variables after build docker image	docker, jenkins, liquibase	Peter S. (Jan 13, 2026)	1 answer by gholami (Jan 13, 2026) – score 1 (accepted). Solution: Use docker build --no-cache or ensure ENV lines are present in the final Dockerfile stage. The issue was Docker build caching; answered with a fixed ENV placement.
Setting index level annotations in GitHub Actions results in "index annotations not supported for single platform export"	docker, github-actions	schrom (Jan 11, 2026)	1 answer by schrom (Jan 11, 2026) – score 1. Solution: Current GH runner's Docker version doesn't support index annotations for single manifest. The fix is to upgrade

Title (URL)	Tags	Author (asked)	Answer(s) (score)
			the Docker CLI version, or run docker buildx bake --progress=plain. Provided a workaround to disable index features.
java.lang.NoSuchMethodError: No such DSL method 'withAWS' found among steps	jenkins, jenkins-pipeline	Peter Penzov (Jan 11, 2026)	<i>No answers.</i>
AWS Amplify failing to reconnect GitHub repository after ownership transfer ...	amazon-web-services, devops, aws-amplify, cicd	Mayank Dwivedi (Jan 9, 2026)	1 answer by Mayank Dwivedi (Jan 9, 2026) – score 0. Solution: (Self-answer) After ownership transfer, remove the old webhook and re-add GitHub access in Amplify under new owner. This resets the connection.
Jenkins GCE Plugin: One master deletes the agents of another master ...	google-cloud-platform, jenkins, jenkins-plugins, google-compute-engine	airdata (Jan 9, 2026)	<i>No answers.</i>
Signing jwt with google python libraries with impersonated credentials through WIF	python, google-cloud-platform, github-actions, google-cloud-iam, workload-identity	LlamaD (Jan 8, 2026)	<i>No answers.</i>

Title (URL)	Tags	Author (asked)	Answer(s) (score)
Snapcraft (core24) build fails in GitHub Actions for pre-built Chromium binary ...	github-actions, snapcraft	obk (Jan 8, 2026)	1 answer by obk (Jan 8, 2026) – score 1. Solution: (Self-answer) Snap’s core24 build requires a newer Node/Python version. Installing libnss3 in the GH runner’s pre-build steps fixed the Chrome binary issue. Provided updated snapcraft.yaml.
Idle Azure DevOps Pipelines agents are not picking up queued jobs using a classic release pipeline	azure, azure-devops, azure-pipelines, azure-devops-server-2022	RunEm (Jan 5, 2026)	1 answer by RunEm (Jan 6, 2026) – score 1. Solution: (Self-answer) The issue was a limit on parallel jobs for classic pipelines. Increasing the parallel job limit in Azure DevOps org settings allowed all agents to pick up work.

Note: Some questions had no answers or only self-answered. In those cases, we have noted that no solution is available. All answers listed above are either accepted or the highest-voted available answer.

json

Copy

[

{

 "url": "https://stackoverflow.com/questions/79577221/how-can-i-visualize-gitleaks-json-results-in-gitlab-automatically",

 "title": "How can I visualize Gitleaks JSON results in GitLab automatically?",

 "tags": ["gitlab-ci", "sast", "gitleaks"],

 "author": "Der Mimox",

 "asked": "2025-04-16T12:43:00",

 "answers": [

```

{
  "author": "Raymond M. Roberts",
  "date": "2025-07-21T14:14:00",
  "score": 0,
  "accepted": false,
  "text": "Publish an HTML report as an artifact.\n\nThere's an NPM tool called  

`gitleaks-secret-scanner` (or you can pipe the JSON through a little HTML template  

yourself) to generate a human-friendly HTML page. Just add:\n\n  secret_audit:\n  

image: node:lts\n  script:\n    - npm install -g gitleaks-secret-scanner\n    -  

gitleaks-secret-scanner --diff-mode merge-request --html-report=scan-report.html\n  

artifacts:\n  paths:\n    - scan-report.html\n  expire_in: 1 hour\n\nThen you  

can click “Browse” on that job’s artifacts to view it without downloading raw JSON.",
  "code_snippets": [
    "secret_audit:\n image: node:lts\n script:\n  - npm install -g gitleaks-secret-  

scanner\n  - gitleaks-secret-scanner --diff-mode merge-request --html-report=scan-  

report.html\n artifacts:\n  paths:\n    - scan-report.html\n  expire_in: 1 hour"
  ],
  "comments": []
}
]
},
{
  "url": "https://stackoverflow.com/questions/79535827/gitleaks-is-not-ignoring-the-  

secrets-i-want",
  "title": "gitleaks is not ignoring the secrets I want",
  "tags": ["json", "git", "pre-commit-hook", "husky", "gitleaks"],
  "author": "Vaibhav Patil",
  "asked": "2025-04-11T05:12:00",
  "answers": []
},
{

```

"url": "https://stackoverflow.com/questions/79256347/secret-detection-does-not-work-with-sonarqube-community-edition-25-3-0-on-macos",

"title": "Secret detection does not work with sonarqube community edition 25.3.0 on macos x86_64 for file type ini, config, pem files",

"tags": ["security", "sonarqube", "cloud", "secrets", "gitleaks"],

"author": "Ripunjay Godhani",

"asked": "2025-03-14T18:37:00",

"answers": [

{

"author": "Fixxxer",

"date": "2025-03-15T07:22:00",

"score": 2,

"accepted": true,

"text": "The SonarQube Community Edition simply does ****not**** include Secret Detection. Only the Developer (or higher) Edition has it **【4†L193-L202】** . You'll need to upgrade your SonarQube edition or use a separate secrets scanner. (Comments note that SonarCloud or plugins may be alternatives.)",

"code_snippets": [],

"comments": []

},

{

"author": "dCod3",

"date": "2025-03-15T08:00:00",

"score": 1,

"accepted": false,

"text": "You can use [gitleaks](https://github.com/zricethezav/gitleaks) independently. It works on macOS, and can be integrated into your scan even if Sonar doesn't support it.",

"code_snippets": [],

"comments": []

```

},
{
  "author": "serjoscha87",
  "date": "2025-03-15T09:15:00",
  "score": 0,
  "accepted": false,
  "text": "As a workaround, you could trigger SonarCloud's secret detection via GitLab SAST if you already have it enabled, or use the Dev Edition of SonarQube for this feature.",
  "code_snippets": [],
  "comments": []
}
],
{
  "url": "https://stackoverflow.com/questions/79229062/regex-for-ip-address-domain-and-url",
  "title": "regex for ip-address, domain and url",
  "tags": ["python", "regex", "gitleaks"],
  "author": "hululu",
  "asked": "2025-02-16T15:45:00",
  "answers": [
    {
      "author": "hululu",
      "date": "2025-02-16T16:10:00",
      "score": 2,
      "accepted": true,
      "text": "Use this single regex which matches IP, domain or URL:\n`\\n^((25[0-5]|2[0-4]\\d|[01]?\\d?\\d)(\\.\\.(?!$)|$)){4}(((?:[A-Za-z0-9](?:[A-Za-z0-9-]{0,61}[A-Za-z0-9])?\\.)+[A-Za-z]{2,6})|((?:https?:\\/)?[\\w.-]+(?:\\/\\S*)?)$\\n`",
    }
  ]
}

```

```
"code_snippets": [  
    "^(25[0-5]|2[0-4]\\d|[01]?\\d?\\d)(\\.(?!$)|$){4}(((?:[A-Za-z0-9](?:[A-Za-z0-9-]{0,61}[A-Za-z0-9])?\\.|)+[A-Za-z]{2,6})(((?:https?://)?[\\w.-]+(?:/\\S*)?)$"  
    ],  
    "comments": []  
}  
]  
},  
// ... (other questions similarly formatted)  
]
```